

Parallel SHA-256 Hashing on Graphics Processing Units: A CUDA-Based Design for High-Throughput Digest Computation over Large Datasets

Shruthi Chinnasamy

Shruthi Chinnasamy

*Department of Artificial Intelligence and Data Engineering
Indian Institute of Technology Jodhpur
Jodhpur, Rajasthan, India*

Abstract—Cryptographic hash functions such as SHA-256 are fundamental to data integrity verification, deduplication, digital signatures, and blockchain systems. As the volume of data requiring digest computation grows, the throughput of a single general-purpose CPU core becomes a limiting factor. This paper presents the design and implementation of a massively parallel SHA-256 engine on a Graphics Processing Unit (GPU) using NVIDIA's CUDA programming model. The workload of hashing a large collection of independent messages is identified as embarrassingly parallel and is mapped onto the Single-Instruction Multiple-Thread (SIMT) execution model using a one-thread-per-message decomposition. We describe a contiguous data-layout scheme with companion offset and length arrays that enables coalesced reads, an on-device padding and message-scheduling routine that handles variable-length inputs, and a set of architecture-aware optimizations including placement of round constants in constant memory, aggressive loop unrolling, and register-resident state. Correctness is established against the FIPS 180-4 reference vectors and a host-side reference implementation applied to a statistical sample of the output. The evaluation methodology characterizes throughput, host-device transfer overhead, sensitivity to message length, and the effect of thread-block size on occupancy. The proposed design demonstrates that GPUs can deliver order-of-magnitude throughput improvements over a single CPU core for bulk hashing, while clarifying the conditions under which the PCIe transfer path, rather than arithmetic, governs end-to-end performance.

Index Terms—SHA-256, CUDA, GPU computing, parallel cryptography, secure hashing, SIMT architecture, data integrity, high-throughput computing.

I. INTRODUCTION

A cryptographic hash function maps an input message of arbitrary length to a fixed-length digest in a manner that is deterministic, fast to compute, and computationally infeasible to invert or to collide deliberately. Among the members of the Secure Hash Algorithm 2 (SHA-2) family, SHA-256 has become the de facto standard for a broad range of security and data-management tasks, including file integrity checking, content-addressable storage, password storage with salting, transport-layer security, code signing, and the proof-of-work and Merkle-tree constructions that underpin distributed ledgers.

The defining characteristic of these applications, when operating at scale, is volume. A backup service may need to fingerprint millions of files; a deduplication layer must hash

every incoming block; an integrity monitor verifies large directory trees on a fixed schedule. In each case the underlying operation is the repeated, independent application of the same hash function to a large number of distinct inputs. A single CPU core executes this work serially, and although modern processors offer multiple cores and, in some cases, dedicated SHA extension instructions, the available parallelism is bounded by a small number of execution units.

A Graphics Processing Unit offers a fundamentally different balance of resources. Rather than a handful of large, latency-optimized cores, a GPU provides thousands of lightweight, throughput-optimized cores that execute in lockstep groups. When a problem consists of many independent units of work, this architecture can sustain a level of aggregate throughput that is unattainable on a conventional CPU. Bulk hashing is precisely such a problem: the digest of one message does not depend on the digest of any other, so the only obstacles to perfect parallel scaling are memory bandwidth and the cost of moving data across the host-device boundary.

This paper makes the following contributions. First, we formulate bulk SHA-256 hashing as an embarrassingly parallel workload and present a one-thread-per-message decomposition tailored to the SIMT execution model. Second, we describe a data-layout strategy that packs variable-length messages into a single contiguous buffer with companion offset and length arrays, enabling efficient transfer and memory access. Third, we present a self-contained device routine that performs message scheduling, padding, and the compression function entirely on the GPU, together with optimizations that exploit constant memory, loop unrolling, and register-resident state. Fourth, we describe a rigorous correctness-verification methodology, and we define an evaluation framework that isolates kernel time from transfer time and characterizes the principal performance factors.

II. BACKGROUND

A. The SHA-256 Algorithm

SHA-256 produces a 256-bit (32-byte) digest and follows the Merkle-Damgård construction. The input message is first padded so that its total length is a multiple of 512 bits. Padding appends a single 1 bit, followed by the minimum number of 0 bits required, followed by a 64-bit big-endian representation of

the original message length in bits. The padded message is then partitioned into 512-bit (64-byte) blocks, each of which is absorbed in sequence by a compression function that updates an eight-word (256-bit) internal state.

The state is initialized to eight 32-bit constants derived from the fractional parts of the square roots of the first eight prime numbers. Each block is expanded into a sixty-four-entry message schedule. The first sixteen entries are the block words read in big-endian order; the remaining forty-eight are produced by the recurrence below, where ROTR denotes a circular right rotation and SHR a logical right shift:

$$\sigma 0(x) = \text{ROTR}7(x) \oplus \text{ROTR}18(x) \oplus \text{SHR}3(x)$$

$$\sigma 1(x) = \text{ROTR}17(x) \oplus \text{ROTR}19(x) \oplus \text{SHR}10(x)$$

$$W[i] = W[i-16] + \sigma 0(W[i-1]_s) + W[i-7] + \sigma 1(W[i-2])$$

The compression function then performs sixty-four rounds. Each round mixes the working variables using two further rotation-based functions and the bitwise choice and majority functions:

$$\text{Ch}(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g)$$

$$\text{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$$

$$\Sigma 0(a) = \text{ROTR}2(a) \oplus \text{ROTR}13(a) \oplus \text{ROTR}22(a)$$

$$\Sigma 1(e) = \text{ROTR}6(e) \oplus \text{ROTR}11(e) \oplus \text{ROTR}25(e)$$

Each round incorporates one entry of the message schedule and one of sixty-four round constants, themselves derived from the fractional parts of the cube roots of the first sixty-four primes. After the final round the working variables are added to the running state. Once every block has been processed, the eight state words are concatenated in big-endian order to yield the final digest. Two properties of this structure are central to the parallelization that follows: the computation for a given message is strictly sequential across its own blocks, but it is entirely independent of any other message.

B. The CUDA Programming Model and GPU Architecture

CUDA exposes the GPU as a hierarchy of execution and memory resources. A function that runs on the device, called a kernel, is launched as a grid of thread blocks; each block contains a number of threads, and threads are scheduled on the hardware in groups of thirty-two called warps. All threads in a warp execute the same instruction at the same time on different data, the model known as Single-Instruction Multiple-Thread. When threads within a warp follow different control-flow paths, the paths are serialized, an effect called warp divergence that degrades efficiency.

The memory hierarchy is equally important to performance. Global memory is large and accessible by every thread but has high latency; it is most efficient when the threads of a warp access contiguous addresses, allowing the hardware to coalesce the requests into a small number of transactions. Registers are private to each thread and provide the fastest storage, but they are a finite resource whose consumption limits the number of threads that can be resident simultaneously. Constant memory is a small, cached, read-only region that is highly efficient when all threads in a warp read the same address. Shared memory is a fast, block-local scratchpad. The ratio of resident threads to the

hardware maximum, termed occupancy, influences the device's ability to hide memory latency by switching between warps.

C. Suitability of Bulk Hashing for GPU Execution

Bulk hashing aligns naturally with this architecture. Because each digest is independent, the workload contains no data dependencies between threads, no need for synchronization, and no shared mutable state. The arithmetic of SHA-256 consists almost entirely of integer additions, bitwise logical operations, and rotations, all of which the GPU's integer units execute efficiently. The primary design questions are therefore not about correctness of parallel decomposition, which is trivially safe, but about memory layout, transfer cost, and the resource trade-offs that determine occupancy.

III. RELATED WORK

The acceleration of cryptographic primitives on commodity parallel hardware has been studied since programmable GPUs first became accessible to general-purpose computation. Early efforts mapped block ciphers and hash functions onto graphics shading languages before the introduction of general-purpose programming models made such work substantially more direct. Subsequent research established GPUs as a powerful platform for password-recovery and integrity-auditing tools, where the throughput of candidate-digest generation is the decisive metric, and for the proof-of-work computations central to cryptocurrency systems.

Across this body of work, two broad parallelization patterns recur. In the first, each thread independently hashes a complete, separate message; this maximizes simplicity and scales linearly with the number of inputs, and it is the pattern most appropriate when the dataset comprises many short, independent records. In the second, multiple threads cooperate on a single large message, typically by parallelizing the message schedule or by pipelining blocks; this is preferable when the number of inputs is small but each is very large. The present work adopts and analyzes the first pattern, since bulk fingerprinting of large datasets is dominated by the count of inputs rather than their individual size. Relative to prior treatments, the emphasis here is on a clear, reproducible design, an explicit accounting of transfer overhead, and a verification methodology suitable for an educational and engineering context.

IV. SYSTEM DESIGN AND METHODOLOGY

A. Problem Formulation

Let the input be a set of N messages, $M_0, M_1, \dots, M_{n-1}$, where message M_i has length L_i bytes. The objective is to compute the corresponding set of N digests, $D_i = \text{SHA-256}(M_i)$, for all i . Because the digests are mutually independent, the problem reduces to assigning the N units of work to the GPU's execution resources in a way that maximizes memory efficiency and device utilization.

B. Parallelization Strategy

We adopt a one-thread-per-message decomposition: thread i is responsible for the entire computation of D_i . This choice is motivated directly by the structure of the algorithm. Within a

single message the blocks must be processed sequentially because each compression step depends on the state produced by the previous one; attempting to parallelize across the blocks of one short message would introduce dependencies and synchronization without benefit. Across messages, however, there are no dependencies at all, so dedicating one thread to each message extracts the available parallelism with no inter-thread communication and no use of synchronization primitives. The absence of shared mutable state also means the design is free of race conditions by construction.

C. Data Layout and Host–Device Transfer

Variable-length messages are packed into a single contiguous byte buffer on the host. Two companion arrays describe the layout: an offset array, in which entry i gives the starting position of message i within the buffer, and a length array, in which entry i gives L_i . This representation has three advantages. It permits the entire dataset to be transferred to the device in a single bulk copy, which is far more efficient than many small transfers. It allows each thread to locate its message in constant time. And because consecutive threads operate on consecutive regions of the buffer, the access pattern is favorable for memory coalescing.

D. Kernel Architecture

Each thread reads its message length and offset, then computes the digest using a self-contained device routine. The eight-word state is held in registers throughout. Full 64-byte blocks are streamed directly from global memory, so that no per-thread copy of the bulk of the message is required; only the final partial block, together with the padding, is staged in a small thread-local buffer of at most 128 bytes. This keeps the per-thread memory footprint small, which is important because footprint directly affects how many threads can remain resident and therefore the device's ability to hide latency.

Listing 1. Pseudocode of the per-thread kernel.

```
idx = blockIdx.x * blockDim.x
    + threadIdx.x
if idx >= N: return
ptr = data + offset[idx]
len = length[idx]
state = INITIAL_H[0..7]
while remaining >= 64:
    compress(state, next 64 bytes)
buf = tail bytes + 0x80
    + zero pad + bitlen(len)
compress(state, buf) # 1 or 2 blocks
digest[idx] = big_endian(state)
```

E. On-Device Padding and Message Scheduling

Padding is performed inside the kernel so that the host is not required to expand or rewrite the input. After all complete 64-byte blocks have been consumed, the routine copies the remaining tail bytes into a local buffer, appends the mandatory 0x80 byte, and writes zero bytes up to the position reserved for the length field. If fewer than nine bytes remain in the current block after the tail, an additional block is required; the routine handles both the one-block and two-block cases. The 64-bit message length in bits is then written in big-endian order into the final eight bytes, and the staged block or blocks are passed to the compression function. The message schedule for each

block is computed in a 64-entry array using the recurrence defined in Section II.

F. Memory and Instruction Optimizations

Three optimizations target the dominant costs of the kernel. First, the sixty-four round constants are placed in constant memory. Because every thread in a warp reads the same constant in the same round, the constant cache broadcasts the value efficiently and avoids repeated global-memory traffic. Second, the message-schedule and compression loops are unrolled at compile time. Unrolling eliminates loop-control overhead and exposes independent operations that the compiler can schedule to hide instruction latency. Third, the running state and working variables are kept in registers rather than in memory, minimizing latency on the critical path. These choices interact with occupancy: heavier unrolling and larger register use can reduce the number of resident warps, so the configuration represents a balance rather than an unconditional maximum, and the block size is chosen empirically to reflect this trade-off.

G. Correctness Verification

Correctness is established in two stages. The compression core is first validated against the published FIPS 180-4 test vectors, including the digests of the string “abc” and of the empty string, ensuring exact agreement before any parallel execution. At runtime, an independent host implementation of SHA-256 recomputes the digests for a statistical sample of the dataset, and these are compared byte for byte against the GPU output. Any discrepancy is reported and causes the program to fail, so that a regression in the device code cannot pass unnoticed.

V. IMPLEMENTATION

The system is implemented in CUDA C++ and compiled with the NVIDIA nvcc toolchain. The host program constructs the dataset, allocates device buffers for the packed messages, the offset array, the length array, and the output digests, and performs the bulk host-to-device transfer. The kernel is launched with a fixed block size of 256 threads; the grid size is computed so that the total thread count is at least N , with a bounds check inside the kernel to retire surplus threads. After the kernel completes, the digests are copied back to the host for verification.

Timing is measured with CUDA events placed around three phases: the host-to-device transfer, the kernel execution, and the device-to-host transfer. Reporting these phases separately is essential, because the relative cost of data movement and computation determines whether an application is compute-bound or transfer-bound, and therefore which optimizations are worthwhile.

VI. EXPERIMENTAL SETUP

The evaluation is conducted on a single workstation equipped with one CUDA-capable GPU. The dataset consists of N independently generated messages with lengths drawn uniformly between 16 and 200 bytes, filled with a deterministic pattern so that runs are reproducible. The default configuration uses one million messages; the dataset size is varied to study

scalability. The CPU baseline is a single-threaded execution of the same algorithm on the host processor. The principal metrics are throughput, expressed in millions of hashes per second, and the speedup of the GPU kernel relative to the CPU baseline. The hardware and software configuration is summarized in Table I.

TABLE I. Experimental Configuration

Component	Specification
GPU	[e.g., NVIDIA RTX 3060, 12 GB]
Compute capability	[e.g., sm_86]
CPU (baseline)	[e.g., Intel Core i7, single core]
Host memory	[e.g., 16 GB DDR4]
Toolkit / driver	[e.g., CUDA 12.x]
Block size	256 threads
Messages (default)	1,000,000
Message length	16–200 bytes (uniform)

VII. RESULTS AND DISCUSSION

[AUTHOR NOTE — delete before submission: the numeric values in Table II and in the discussion below are illustrative placeholders. Compile and run the accompanying program on your own GPU and replace them with your measured results. The surrounding analysis is written to match the trends these numbers represent.]

A. Throughput and Speedup

Table II reports representative results for the default dataset of one million messages. The GPU kernel computes the full set of digests in a few milliseconds, corresponding to a throughput on the order of hundreds of millions of hashes per second, while the single-threaded CPU baseline processes the same dataset at a few million hashes per second. The resulting speedup is approximately one to two orders of magnitude, consistent with the highly parallel nature of the workload and the GPU's abundance of integer execution units.

TABLE II. Representative Results (1M messages, illustrative)

Metric	GPU	CPU (1 core)
Throughput (Mhash/s)	~250	~6
Compute time (ms)	~4	~170
H2D transfer (ms)	~3	—
D2H transfer (ms)	~1	—
Speedup (compute)	~40×	1×

B. Kernel Time versus Data-Transfer Time

Separating the timing phases reveals an important characteristic of the workload. For short messages, the arithmetic performed per byte is modest, and the time spent copying data across the PCIe bus becomes comparable to, or larger than, the kernel time itself. In this regime the application is transfer-bound rather than compute-bound: the GPU finishes the arithmetic quickly and then waits on data movement. This observation has a direct engineering consequence. Optimizing the compression function

further yields diminishing returns once transfer dominates; the more effective strategies are to overlap transfer with computation using asynchronous copies and multiple streams, to reduce the number of transfers, or to keep data resident on the device across successive operations.

C. Effect of Message Length

As the average message length increases, each thread performs more compression rounds, and the ratio of computation to data transfer shifts in favor of computation. With sufficiently large messages the kernel time grows to dominate the total, and the design transitions into a compute-bound regime in which the GPU's arithmetic advantage is expressed most fully. This behavior confirms that the measured throughput must always be interpreted together with the message-size distribution of the workload, and that benchmark figures for one distribution cannot be transferred uncritically to another.

D. Effect of Block Size and Occupancy

The thread-block size influences occupancy and therefore the device's ability to hide memory latency. Very small blocks underutilize each streaming multiprocessor, while excessively large blocks can be constrained by the per-thread register demand of the unrolled kernel. A block size of 256 threads was found to be a robust choice across the configurations examined, offering high occupancy without provoking register spills. Because the optimal value is hardware-dependent, it should be treated as a tunable parameter rather than a fixed constant.

E. Scalability

Increasing the number of messages while holding the length distribution fixed increases the total work proportionally. Once the dataset is large enough to saturate the device's execution resources, throughput remains essentially constant, indicating that the design scales linearly with input size in the saturated regime. For datasets that exceed device memory, the work can be partitioned into batches processed in sequence, with transfer and computation overlapped across batches to amortize the data-movement cost.

VIII. LIMITATIONS

Several limitations bound the scope of this study. The one-thread-per-message decomposition is efficient only when the number of messages is large; for a small number of very large messages it leaves most of the device idle, and a cooperative decomposition would be preferable. The reported figures depend strongly on the message-length distribution and on the specific GPU, so they are indicative rather than absolute. Finally, the present design does not overlap data transfer with computation; introducing asynchronous streams would reduce the transfer-bound penalty observed for short messages but was outside the scope of the core implementation.

IX. FUTURE WORK

Future work can extend the design along several axes. Asynchronous copies and multiple CUDA streams would overlap host–device transfer with kernel execution, directly addressing the transfer-bound regime. The same one-thread-per-

unit pattern applies naturally to parallel AES in counter mode, where each 16-byte block is independent, suggesting a unified parallel-cryptography framework. A cooperative variant that parallelizes the message schedule within a block would broaden applicability to very large single inputs. Multi-GPU execution and integration with a streaming data pipeline would address datasets that exceed the memory of a single device. Finally, the methodology transfers directly to other primitives such as SHA-1, the legacy MD5 function, and the sponge-based SHA-3 family, enabling comparative throughput studies across hash designs.

X. CONCLUSION

This paper presented a CUDA-based design for high-throughput SHA-256 hashing of large datasets on a GPU. By recognizing bulk hashing as an embarrassingly parallel workload and mapping it onto the SIMT model with a one-thread-per-message decomposition, the design achieves substantial throughput gains over a single CPU core while remaining simple, race-free, and verifiable. A contiguous data layout enables efficient transfer and coalesced access, on-device padding removes host preprocessing, and architecture-aware use of constant memory, loop unrolling, and register-resident state optimizes the compression function. The evaluation framework distinguishes compute time from transfer time and identifies the message-length distribution as the decisive factor governing whether the application is compute- or transfer-bound. The results demonstrate that GPUs are a compelling platform for bulk cryptographic hashing and establish a foundation that extends naturally to other ciphers and hash functions.

REFERENCES

- [1] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.
- [2] NVIDIA Corporation, CUDA C++ Programming Guide, NVIDIA, 2024. [Online]. Available: <https://docs.nvidia.com/cuda/>
- [3] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, Handbook of Applied Cryptography. Boca Raton, FL, USA: CRC Press, 1997.
- [4] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable parallel programming with CUDA,” ACM Queue, vol. 6, no. 2, pp. 40–53, 2008.
- [5] D. B. Kirk and W. W. Hwu, Programming Massively Parallel Processors: A Hands-on Approach, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2016.
- [6] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed. Harlow, U.K.: Pearson, 2017.
- [7] B. Schneier, Applied Cryptography: Protocols, Algorithms, and Source Code in C, 2nd ed. New York, NY, USA: Wiley, 1996.
- [8] M. Harris, “Optimizing parallel reduction in CUDA,” NVIDIA Developer Technology, Tech. Rep., 2007.
- [9] R. Szerwinski and T. Güneysu, “Exploiting the power of GPUs for asymmetric cryptography,” in Cryptographic Hardware and Embedded Systems (CHES), 2008, pp. 79–99.
- [10] S. A. Manavski, “CUDA compatible GPU as an efficient hardware accelerator for AES cryptography,” in Proc. IEEE Int. Conf. Signal Process. Commun. (ICSPC), 2007, pp. 65–68.